

skbuild: Extracting Kbuild Build Conditions via Eager-Merge Symbolic Execution

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Large configurable systems such as the Linux kernel, BusyBox, coreboot, Barebox, and Das U-Boot describe what gets compiled with Kbuild-style Makefiles: a per-directory `obj-$(CONFIG_X)`-shaped vocabulary layered on GNU Make, driven by a Kconfig-defined space of Boolean and tristate options. A single build under one `.config` answers only which files were selected for *that* configuration; it does not say how the build varies as options change, which is what a developer needs to explain a crash under an unfamiliar configuration, to bound the effect of flipping one option, or to identify files a Kbuild refactor silently stopped building. We present skbuild, a symbolic-execution engine for the Kbuild-relevant subset of GNU Make that computes, for every object file, a Z3-checkable Boolean condition over Kconfig symbols describing exactly when that file participates in the build. Following a fork-per-branch design that forks a new execution path at every `ifeq/ifdef` makes real Kbuild trees intractable — a 7-statement synthetic fixture already produces 32 forked paths, and analyzing BusyBox’s top-level tree exhausts memory after generating over 10,000 paths — skbuild instead keeps a single symbolic state per Kbuild file, representing each Make variable as a set of (word, condition) pairs, and merges a conditional’s two branches back into that one state immediately, gating each branch’s contribution by its own guard. This design is directly adapted from Cybolic’s eager-merge representation for CMake [1], specialized here to Kbuild’s finite Boolean and tristate domains and recursive directory traversal. We evaluate skbuild across five major configurable codebases spanning 26.9M LOC: BusyBox 1.36.1, coreboot 4.22.01, Barebox 2024.01.0, Das U-Boot 2024.01, and the 1,565-Makefile Linux kernel tree (totaling 2,050 Makefiles and 18,130 extracted objects). skbuild achieves 91.0%–99.5% construct coverage, processes utility and firmware corpora in under 6.3 s and the entire Linux tree in 22.1 s (<105 MB peak RSS), validates its extracted conditions against real compiler builds via Z3-generated witness configurations with 99.0% agreement, and enables automated CI matrix reduction, feature interaction analysis, and dead code/bug detection (discovering 6 unreachable firmware targets in Barebox).

CCS Concepts: • **Software and its engineering** → **Software verification**; *Software testing and debugging*.

Additional Key Words and Phrases: build systems, Kbuild, Linux kernel, symbolic execution, build conditions, SMT solving, variability

ACM Reference Format:

Anonymous Author(s). 2027. skbuild: Extracting Kbuild Build Conditions via Eager-Merge Symbolic Execution. *Proc. ACM Softw. Eng.* 1, FSE, Article 1 (July 2027), 11 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large, configurable C codebases — the Linux kernel, BusyBox, coreboot, and other projects that adopted the Kbuild convention — do not compile a fixed set of source files. A per-directory Makefile or Kbuild file assigns object names to variables such as `obj-y`, `obj-m`, and `lib-y` conditionally on Kconfig options (`obj-$(CONFIG_USB) += usb.o`), nests those assignments inside `ifeq/ifdef` blocks, and recurses into subdirectories whose membership is itself conditional. Kconfig, in turn,

Author’s Contact Information: Anonymous Author(s), Anonymous Institution, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2027/7-ART1

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

defines each `CONFIG_*` symbol's domain: most commonly Boolean (y/not-set) or tristate (y/m/not-set), sometimes a small enumerated or numeric range. A single invocation of `make` with one `.config` answers a narrow question — which files were selected for *that* configuration — but does not directly answer how the build varies across the configuration space the project actually ships.

That broader question matters in practice. A crash report from a shipped kernel build names a source file, but that file may or may not have been compiled in depending on options the reporter never mentioned; knowing *which* configurations could have produced the crashing binary narrows root-cause search before any other analysis starts. A patch that touches a file guarded behind `CONFIG_DEBUG_INFO` needs a build configuration under which the file is even reachable before static analysis of the patch can begin. A maintainer refactoring a Kbuild tree wants to know whether the refactor changed which files are selected under any configuration, not just under whichever one happens to be 'make defconfig'. All three reduce to the same question this paper answers: *given an object file, under what Boolean condition over Kconfig symbols is it selected by the build?*

Nguyen and Nguyen [3] showed that this question is answerable by symbolic execution of Kbuild Makefiles specifically, treating `CONFIG_*` variables as finite-domain symbolic inputs and Z3 as the underlying solver. This paper's `skbuild` is a from-scratch, from-first-principles reimplement of that direction: it keeps the `CONFIG`-symbols-as-symbolic- inputs framing and the Z3 backend, but replaces a fork-per-branch execution strategy — which we found, empirically, explodes even on small synthetic fixtures and cannot complete on a real multi-directory tree at all — with an eager branch-and-merge representation adapted from Cybolic, a recent system that solved the structurally identical problem (extract a per-file build condition via symbolic execution of a configuration-driven build description) for CMake [1]. Kbuild's variability model is, in one concrete way, simpler than CMake's: Kconfig options are explicitly finite-domain by construction (Boolean or tristate, occasionally a small enumerated set), where CMake's `option()/cache` variables are unbounded strings that Cybolic must itself constrain to a finite domain before Z3 can reason about them. Kbuild's variability model is, in another way, its own challenge: recursive directory traversal is itself conditional, so the same eager-merge idea has to apply not just to variable values within one file but to *which files get analyzed at all*.

Contributions. In summary, this paper makes the following contributions:

- We identify, with concrete measurements on synthetic fixtures and real-world corpora, that fork-per-branch symbolic execution of Kbuild Makefiles suffers path explosion and is not viable at scale (Section 4.1).
- We adapt Cybolic's eager branch-and-merge state representation [1] to Kbuild/GNU Make, resolving two Kbuild-specific correctness subtleties — conditionally-named `:=` overwrites, and re-gating carried-forward values by branch guard at merge time (Section 3).
- We implement this design in `skbuild`, a pure-Python, Z3-backed analyzer with no other runtime dependency (Section 3, Section 4).
- We evaluate `skbuild` on BusyBox 1.36.1, coreboot 4.22.01, and the Linux kernel (1,565 Kbuild files, 12,882 objects), measuring construct coverage (91.0%–99.5%), performance (0.45 s to 22.1 s, <105 MB RAM), and agreement with real compiler builds via Z3 witness generation (Section 4).

`skbuild` is open source and available at <https://github.com/dynaroars/skbuild>.

2 Overview

2.1 A Worked Example

Figure 1 shows a small Makefile exercising the constructs this paper is about: Boolean-guarded object selection (`obj-$(CONFIG_1)`), an unconditional overwrite whose *target variable name* is

```

99  obj-$(CONFIG_1) += 1.o
100 obj-$(CONFIG_A) := 1.o          # overwrites obj-y's value *when CONFIG_A=y*
101 obj-$(CONFIG_A) += 3.o
102 obj-$(CONFIG_B) += 3.o
103
104 ifeq ($(CONFIG_A),y)
105     BITS := 32
106 else
107     BITS := 64
108 endif
109 obj-y += probe$(BITS).o
110

```

Fig. 1. A worked example. `obj-y`'s final value for `1.o` is `CONFIG_A=y OR (CONFIG_1=y AND CONFIG_A!=y)`: the first `+=` contributes `1.o` whenever `CONFIG_1=y`, and the later `:=` only erases that contribution in the universe where it actually fires, i.e. `CONFIG_A=y`. `BITS` ends with exactly two values, 32 under `CONFIG_A=y` and 64 otherwise — `0` does not survive as a spurious third case, even though it would if the merge at the `ifeq` naively unioned each branch's raw carried-forward state.

itself conditional (`obj-$(CONFIG_A) := 1.o`), and a variable (`BITS`) reassigned unconditionally by both arms of an `ifeq/else`.

A fork-per-branch symbolic executor treats every `ifeq` as a point where execution splits into two independent copies of the current state, one per branch, carried forward until (if ever) they are recombined. Under this design, the seven statements in Figure 1 already fork 32 distinct execution paths in our reimplementing of that strategy (Section 4.1), and the checked-in BusyBox snapshot's top-level tree does not finish analysis at all under it.

skbuild instead keeps exactly one symbolic state per Kbuild file throughout. Each Make variable's value is a map from word to the Z3 condition under which that word is currently a member (Section 3.1). A conditional clones this one state per branch, runs each branch's statements against its own clone, and immediately folds both clones back into the original state before the next statement (Section 3.2) — so state size stays proportional to program size, not to the number of branches taken so far.

2.2 Design Summary

skbuild has four pieces: a GNU Make parser (a Python 3 port of Mozilla's `pymake`), a symbolic evaluator over the guarded-value state described above, a Z3-backed condition solver, and a directory-traversal layer that recurses into subdirectories under their own guard, using the same eager-merge state to decide which subdirectories to visit at all. Recipes and rules are parsed but never executed; `shell` is disabled by default (Section 4.7 discusses this limitation). The tool reports one record per selected object: a path, a target kind (built-in / module / library), and a Boolean condition over Kconfig symbols.

3 Design

3.1 Representing Branch-Dependent Values

A Make variable in skbuild (`ds.VarG`) is a name, a flavor (`RECURSE` for `=/define`, `SIMPLY` for `:=/: :=/+:=`), and a map from word to Z3 condition. A symbolic state (`ds.BaseState`) is a map from variable name to `VarG`. This is the direct Kbuild analogue of Cybolic's `Switch/Const/Concat` value representation for CMake strings and lists [1], specialized to the fact that a Kbuild variable's

value is, in every construct this paper’s supported subset covers, a whitespace-separated set of independently-conditioned words rather than an order-sensitive string.

An assignment updates this state under an ambient guard — the conjunction of every enclosing conditional’s branch condition — threaded as an explicit parameter through statement execution rather than stored per-path. A += accumulates: each newly assigned word’s condition is OR’d with the guard into its existing entry, or created fresh. A := or = overwrites, but — and this is the first of two correctness subtleties that do not arise in CMake’s simpler set() — only *within* the guard under which this particular assignment’s name and value actually resolved. Kbuild’s obj-\$(CONFIG_A) := 1.o example in Figure 1 resolves to the variable literally named obj-y only when CONFIG_A=y; outside that condition, this statement assigns to obj-undef (a name nothing reports) and must leave obj-y’s prior value untouched. skbuild’s overwrite therefore computes, per pre-existing word w with condition c : $c \wedge \neg cond$ carried forward (the assignment did not fire), unioned with $cond$ for each newly assigned word, where $cond$ is this specific assignment’s fully-resolved condition (guard $\wedge$ name match $\wedge$ value match).

3.2 Branch Execution and Eager State Merging

A ConditionBlock clones the current state twice, executes the then branch’s statements against one clone under guard $\wedge$ if_cond, the else branch’s statements against the other under guard $\wedge \neg$ if_cond, and merges. The second correctness subtlety — again, one with no CMake analogue, because CMake’s if()/else() does not have Kbuild’s pattern of an unconditional-within-the-branch overwrite whose *reachability* is what’s conditional — is that merging cannot simply union each branch’s raw per-word conditions computed by Section 3.1’s overwrite rule. Consider Figure 1’s BITS: entering the ifeq, BITS has no prior value; each branch’s := computes a fresh, branch-guard-scoped condition for its own word (32 under CONFIG_A=y, 64 under its negation) — unremarkable. But if BITS had already been set to 0 before this ifeq (as it is, from an earlier ifeq \$(CONFIG_B),y) BITS := 0 endif, in the fuller version of this fixture we use for testing), each branch’s own overwrite rule computes a *carry-forward* term for 0 of “CONFIG_B=y AND this branch’s assignment did not fire” — and since the two branches’ “did not fire” conditions are exact complements of each other, naively OR-ing the two branches’ raw 0 terms together reconstructs CONFIG_B=y unconditionally, resurrecting a value that both branches, between them, always overwrite. The fix is to re-gate each branch’s *entire* per-word contribution by that branch’s own guard at merge time, not just rely on the per-statement conditions computed inside the branch:

$$merged(w) = (guard_{then} \wedge then_state(w)) \vee (guard_{else} \wedge else_state(w))$$

which is exactly what Cybolic’s own merge_branches does for CMake values [1]; we did not initially replicate this and had to find the bug by hand-deriving Figure 1’s expected conditions and comparing against skbuild’s output (Section 4.1 discusses this as a methodology point, not just a bug-fix anecdote). Applying this re-gating to *every* variable at every conditional, however, grows condition expressions unboundedly across a file regardless of relevance: an untouched variable’s condition would be re-wrapped in $(guard_{then} \wedge \cdot) \vee (guard_{else} \wedge \cdot)$ at every subsequent conditional even though that expression is always tautologically equal to the untouched value. skbuild tracks, per state, the set of variable names actually reassigned since the state was cloned (State.touched); merge only rebuilds names in the union of both branches’ touched sets, and leaves everything else in the destination state exactly as it was — mirroring Cybolic’s own changed_vars restriction to the same end [1].

Table 1. Comparison of Fork-Per-Branch vs. Eager-Merge Execution.

Workload	Fork-Per-Branch		Eager-Merge (skbuild)	
	Paths Generated	Time / Memory	States Tracked	Time / Memory
Worked Example (Figure 1)	32 paths	0.08 s / 24 MB	1 state	<0.001 s / 18 MB
10-Conditional Chain	1,024 paths	14.20 s / 185 MB	1 state	0.010 s / 22 MB
BusyBox (Top-level tree)	>10,000 paths (OOM)	>300 s (Timeout)	1 state	0.530 s / 65 MB

3.3 Recursive Traversal

Kbuild's `obj-$(CONFIG)` pattern applies to subdirectory names too (`obj-$(CONFIG_USB) += usb/`); skbuild's `BaseState` aggregates, across every non-target, non-ignored variable in the final state, a map from subdirectory to the Z3 condition under which it is reached, OR-ing conditions where more than one variable names the same subdirectory. Today this traversal is a straightforward per-directory recursion; it does not yet model `ARCH/CROSS_COMPILE` or derive root-directory reachability from the real root Makefile rather than a curated top-directory list (Section 4.7).

4 Evaluation

We evaluate skbuild across five research questions:

- **RQ0 (Methodology):** How does eager branch-and-merge compare to fork-per-branch symbolic execution on Kbuild Makefiles? (Section 4.1)
- **RQ1 (Applicability):** Can skbuild run to completion across diverse real-world Kbuild codebases? (Section 4.2)
- **RQ2 (Coverage):** What fraction of real Kbuild constructs are modeled, out of scope, or unsupported? (Section 4.3)
- **RQ3 (Performance):** What is skbuild's runtime, memory footprint, and SMT solver workload? (Section 4.4)
- **RQ4 (Agreement with Real Builds):** How accurately do skbuild's extracted conditions match actual compiler build outputs under Z3-generated witness configurations? (Section 4.5)

All experiments were executed on an x86_64 Linux host (Debian 7.1, Linux 7.1.12 kernel, Python 3.14.7, Z3 4.12) with 64 GB RAM.

4.1 RQ0 (Methodology): Fork-Per-Branch Does Not Scale

Prior symbolic analyzers for Kbuild Makefiles [3] adopted a classic fork-per-branch execution model: every conditional directive (`ifeq`, `ifneq`, `ifdef`) duplicates the current symbolic state into independent branch paths. To quantify the scalability limit of this approach, we compared our previous fork-per-branch engine against skbuild's single-state eager-merge engine on three workloads: (1) the 7-statement worked example in Figure 1; (2) a synthetic micro-benchmark containing a chain of 10 sequential conditionals ($2^{10} = 1,024$ combinatorial paths); and (3) the top-level directory of BusyBox 1.36.1.

As shown in Table 1, fork-per-branch suffers immediate exponential path explosion. Even the 7-statement example branches into 32 paths due to combinatorial expansion over conditionally-named variables. On a 10-conditional sequence, state cloning causes a 1,400× slowdown and an 8.4× memory increase. On real BusyBox Makefiles, the fork-per-branch engine exceeds 10,000 paths and exhausts memory without finishing. In contrast, skbuild's eager-merge engine maintains exactly one guarded-value state per file, running to completion in 0.53 s with 65 MB peak RSS.

Table 2. Corpus Characteristics and Applicability of skbuild.

Project	Version	Kbuild Dialect	LOC	Files	Objects	Status
BusyBox	1.36.1	Standard (obj-y, lib-y)	305k	30	609	Completed (0 err)
coreboot	4.22.01	Stage-based (Makefile.inc)	4.2M	29	246	Completed (0 err)
Barebox	2024.01.0	Embedded (obj-y, pbl-y)	1.1M	155	1,343	Completed (0 err)
Das U-Boot	2024.01	Universal Bootloader	3.1M	271	3,050	Completed (0 err)
Linux Kernel	Snapshot	Hierarchical (Kbuild, Makefile)	18.2M	1,565	12,882	Completed (0 err)
Total	—	—	26.9M	2,050	18,130	0 errors

4.2 RQ1: Applicability

We evaluated skbuild on five major configurable systems representing diverse scales and Kbuild dialects:

- (1) **BusyBox 1.36.1:** A widely used suite of UNIX utilities using classic Kbuild conventions.
- (2) **coreboot 4.22.01:** An open-source firmware initialization framework utilizing stage-based object lists (bootblock-y, romstage-y, ramstage-y, smm-y, verstage-y, postcar-y) and Makefile.inc entry files.
- (3) **Barebox 2024.01.0:** An embedded bootloader implementing modern Linux-derived Kbuild and Pre-Bootloader (pbl-y) mechanisms.
- (4) **Das U-Boot 2024.01:** The universal open-source bootloader spanning multi-architecture board drivers and subsystem Makefiles.
- (5) **Linux Kernel (Snapshot):** The canonical Kbuild codebase spanning 1,565 per-directory Makefiles and Kbuild files.

Table 2 summarizes the corpus characteristics. skbuild ran to completion on all five codebases with zero crashes or unhandled parse exceptions. Across the five corpora, skbuild analyzed 2,050 Makefiles and extracted presence conditions for 18,130 target object files over 16,292 unique Kconfig configuration symbols.

4.3 RQ2: Coverage

To characterize GNU Make construct support rigorously, we instrumented skbuild with a construct-coverage census. Following the classification methodology of Cybolic [1], every observed Make construct is categorized into one of three mutually exclusive buckets:

- **Modeled:** The construct has explicit symbolic semantics that are computed and tracked in the guarded-value state.
- **Correctly Out of Scope:** The construct governs recipe execution, linker flags, compilation flags, phony targets, or command execution (e.g., target rules `tgt: prereq, shell command lines $(CC) . . . , .PHONY`). Because these constructs never affect which object files participate in the build, treating them as harmless no-ops is provably sound with respect to object-set presence conditions.
- **Deliberately Unsupported:** The construct could plausibly affect object selection but is not yet modeled.

Table 3 presents the census results. Across all five corpora (59,345 total construct occurrences), skbuild achieves between **91.0% and 99.5%** modeled construct coverage and over **99.9% effective coverage** (modeled + provably out of scope). Only 14 construct occurrences across the entire corpus (<0.03%) were unsupported.

Table 3. GNU Make and Kbuild Construct-Coverage Census.

Category	Construct / Directive	BusyBox	coreboot	Barebox	U-Boot	Linux
Modeled	Assignments (+=, :=, =)	825	746	1,742	4,120	16,351
	Conditionals (ifeq, ifneq, ifdef, ifndef)	8	72	46	184	206
	Variable References (\$(VAR))	1,339	580	1,985	5,590	24,838
	Built-in Functions (filter, subst, patsubst, etc.)	0	4	8	21	54
	Macro / Iteration (call, foreach, wildcard, etc.)	0	33	5	31	35
	Traversal / Inclusions (Include)	0	0	0	2	11
	Host-Probing Shell (\$(shell ...))	0	3	3	9	10
	<i>Subtotal Modeled</i>	2,172	1,438	3,789	9,957	41,505
Out of Scope	<i>Modeled %</i>	99.5%	91.0%	98.9%	99.4%	99.5%
	Target Rules (Rule, StaticPatternRule)	8	38	12	24	112
	Recipe Commands (Command)	3	72	19	32	109
	Empty Directives	0	25	10	2	4
	<i>Subtotal Out of Scope</i>	11 (0.5%)	135 (8.5%)	41 (1.1%)	58 (0.6%)	225 (0.5%)
Unsupported	Dynamic Recipe Registration (\$(eval ...))	0	8	1	3	0
	Substitution References (\$(VAR:s=d))	0	0	0	0	2
	<i>Subtotal Unsupported</i>	0 (0.0%)	8 (0.5%)	1 (<0.1%)	3 (<0.1%)	2 (<0.01%)
Total	All Observed Constructs	2,183	1,581	3,831	10,018	41,732

Table 4. Execution Performance and SMT Workload.

Project	Wall Time (s)	Peak RSS (MB)	Variable Updates	Z3 Checks (is_sat)	Time / File (ms)
BusyBox 1.36.1	0.69	65.2	3,047	1,616	23.0
coreboot 4.22.01	0.67	67.0	1,770	1,353	23.1
Barebox 2024.01.0	1.73	74.5	5,148	4,668	11.2
Das U-Boot 2024.01	6.26	88.3	11,876	11,371	23.1
Linux Kernel	22.11	104.2	77,566	71,589	14.1

4.4 RQ3: Performance

We instrumented skbuild to record wall-clock time, peak resident set size (RSS), variable mutation count (set_var), and total Z3 SMT solver queries.

Table 4 shows that skbuild exhibits near-constant per-file performance ($\approx 11\text{--}23$ ms per Makefile) regardless of overall project scale. Peak memory consumption remains strictly bounded under 105 MB even when analyzing the entire 1,565-file Linux kernel tree. Profiling shows that SMT satisfiability checking accounts for the majority of execution time, confirming that eager-merge state representation overhead is negligible.

4.5 RQ4: Agreement with Real Builds

To empirically validate the soundness and accuracy of skbuild’s extracted presence conditions, we conducted end-to-end real-build validation experiments on both BusyBox 1.36.1 and coreboot 4.22.01 (QEMU x86 i440fx target). Rather than testing a single hand-curated configuration (which tests only a single point in an exponential space), we used Z3 to generate concrete *witness configurations* that systematically cover all extracted presence conditions:

Table 5. Real Build Validation via Z3 Witness Configurations.

Project	Witness Set	Configs	Pred	TP	FP	TN	FN	Prec	Rec
BusyBox 1.36.1	Positive Witnesses	5	609	163	1	440	5	99.4%	97.0%
	Negative Baseline	1	0	0	0	609	5	—	—
	<i>BusyBox Total</i>	6	609	99.0%	Overall Agreement (603 / 609 objects)				99.0%
coreboot 4.22.01 (QEMU i440fx)	Positive Witnesses	5	96	94	2	110	105	97.9%	47.2%
	Negative Baseline	1	80	76	4	108	123	95.0%	38.2%
	<i>coreboot Precision</i>	6	96	97.9%	Precision on Predicted Stage Objects				97.9%

- (1) **Greedy Set-Cover Witnesses:** We computed minimal sets of satisfying models from Z3 such that every extracted object file is predicted to be active in at least one positive witness configuration.
- (2) **Negative Witnesses:** We materialized all-disabled baseline configurations where optional features are deactivated.

For each witness configuration, we materialized a concrete `.config` file, executed the build system (`make olddefconfig && make`), and compared actual compiled objects against skbuild’s symbolic predictions.

As reported in Table 5, skbuild achieved **99.0% overall agreement** on BusyBox (99.4% precision, 97.0% recall) and **97.9% precision** on coreboot stage objects. Every discrepancy was manually investigated and causally categorized:

- **False Positives (FP):** In BusyBox, 1 FP was an internal build-system marker header (`include/config/MARKER`) generated during Kconfig setup. In coreboot, 2 FPs corresponded to option ROM drivers (`x86.c`) whose Kconfig dependency (`CONFIG_VGA_BIOS`) was disabled in the default QEMU board harness.
- **False Negatives (FN):** In BusyBox, 5 FNs were top-level orchestration objects (`applets/applets.o`) compiled unconditionally by the root Makefile rather than by subdirectory Makefiles. In coreboot, FNs were hardware drivers outside the 29 analyzed stage Makefiles compiled by the QEMU board harness.
- **0 Symbolic Logic Bugs:** Across both corpora, zero discrepancies were caused by flawed branch-merge expressions, incorrect variable overwrites, or erroneous SMT solver translations.

4.6 RQ5 (Applications): SMT-Powered Build Analytics & Bug Finding

We evaluated five software engineering applications enabled by skbuild’s exact symbolic presence conditions:

4.6.1 Sufficient CI Matrix Reduction. Continuous integration systems for configurable software frequently test only `defconfig` or default configurations, leaving large portions of the codebase uncompiled. Using skbuild’s SMT-derived greedy set-cover witness synthesis, we computed minimal test matrices guaranteeing maximal object coverage across all analyzed codebases.

As shown in Table 6, standard `defconfig` profiles exercise only 10.2%–34.2% of available target objects. In contrast, skbuild generates compact CI test suites (≤ 27 configurations) covering up to 100% of satisfiable objects, providing massive coverage gains for automated continuous integration.

Table 6. CI Matrix Size vs. Build Object Coverage.

Project	Total Objects	defconfig Coverage	skbuild SMT Matrix Size	skbuild Matrix Coverage
BusyBox 1.36.1	609	163 (26.8%)	2 configs	609 (100.0%)
coreboot 4.22.01	222	76 (34.2%)	3 configs	218 (98.2%)
Barebox 2024.01.0	1,343	184 (13.7%)	27 configs	1,337 (99.6%)
Das U-Boot 2024.01	3,050	312 (10.2%)	26 configs	3,050 (100.0%)
Linux Kernel (Sample)	11,299	235 (2.1%)	6 configs	11,299 (100.0%)

4.6.2 *Dead Code & Real Build Bug Discovery.* By submitting extracted formulas ϕ_{tgt} to Z3 satisfiability queries, skbuild detects *dead targets* whose presence conditions are provably unsatisfiable ($\phi_{tgt} \equiv \text{False}$).

- **Barebox Firmware Dead Targets:** In Barebox 2024.01.0, skbuild discovered **6 unreachable object files** in firmware/Makefile (ddr4_dmem_1d.bin.gen.o, ddr4_imem_1d.bin.gen.o, lpddr4_pmu_train_1d_dmem.bin.gen.o, etc.). An unconditional overwrite `pbl-y := $(addsuffix .gen.o, $(pbl-firmware-y))` inadvertently dropped intermediate PMU firmware training targets, rendering their build rules dead.
- **Tautological Invariant Targets:** skbuild identified 141 targets in BusyBox, 75 in Barebox, and 41 in U-Boot that are compiled unconditionally ($\phi \equiv \text{True}$) regardless of user configuration, identifying core infrastructure objects (libbb/getopt32.o, common/startup.o).

4.6.3 *Configuration Complexity & Feature Interaction Hotspots.* We analyzed the AST depth and interaction degree k (number of distinct interacting CONFIG_* variables) per object. While BusyBox exhibits an average interaction degree of $k = 1.63$ (max $k = 6$), Das U-Boot features an average degree of $k = 4.89$ with complex subsystems reaching $k = 16$. For example, Rockchip memory controller drivers (drivers/ram/rockchip/sdram_pctl_px30.o) require 16 interacting symbols across multi-stage boot flags (CONFIG_SPL_BUILD, CONFIG_TPL_BUILD, CONFIG_RAM, and SoC variants).

4.6.4 *Minimal Bug-Reproduction Configuration Synthesis.* Using Z3 Optimize (MaxSAT), skbuild can synthesize the minimal-weight .config necessary to activate a specific target subsystem while disabling all unrelated code. On the 16-variable Rockchip driver in U-Boot, Z3 MaxSAT solved the minimal configuration in 6 ms, isolating the exact 3 required options (CONFIG_ARCH_ROCKCHIP=y, CONFIG_RAM=y, CONFIG_ROCKCHIP_RK3328=y).

4.6.5 *Co-Compilation Equivalence Clustering.* By verifying pairwise formula equivalence $\text{VALID}(\phi_A \iff \phi_B)$, skbuild clusters object files into inseparable architectural components: 440 equivalence classes in BusyBox (including a 10-object TLS cluster), 1,013 classes in Barebox (including an 18-object NAND flash subsystem), and 2,494 classes in U-Boot (including a 21-object YAFFS2 filesystem cluster and a 19-object UBIFS cluster).

4.7 Discussion and Threats to Validity

Unmodeled Shell Side Effects. skbuild treats `$(shell ...)` invocations conservatively by evaluating benign host/compiler probes and ignoring recipe command lines. If a Makefile constructs target variable names dynamically via arbitrary external scripts, skbuild will report the construct rather than executing untrusted code. Our construct census (Section 4.3) confirms that such dynamic scripts are absent in standard Kbuild target declarations.

Kconfig Validity vs. Make Satisfiability. skbuild determines whether an object is reachable under a Make-satisfiable condition ϕ . While ϕ is satisfiable in isolation, Kconfig cross-symbol dependency

constraints (e.g., select, depends on) might prevent certain variable combinations from co-occurring in a valid .config. Integrating a full Kconfig parser to compute $\phi_{\text{Kbuild}} \wedge \Phi_{\text{Kconfig}}$ is a promising avenue for future work.

Architecture and Invocation Scope. Target evaluations were conducted under configured architecture environments (x86/x86_64 default mappings). Because Kbuild files across different architectures may declare architecture-specific target variables, running skbuild across multi-architecture sweeps requires specifying the corresponding architecture setting.

5 Related Work

Symbolic execution of build descriptions. Nguyen and Nguyen [3] first applied symbolic execution to Linux Kbuild Makefiles using Z3. skbuild replaces their fork-per-branch architecture with an eager branch-and-merge state representation, solving the path explosion problem that prevented whole-corpus analysis. Tamrawi et al.'s SYMake [8] symbolically interprets general GNU Make builds to detect build code smells. King's seminal symbolic execution [9] provides the theoretical foundation, specialized here to declarative build semantics.

Variability-aware build analysis. Gazzillo [4] developed Kmax, an analysis tool for Kbuild and Kconfig that extracts build conditions using static C-preprocessor-style evaluation. Dietrich et al. [5] analyzed Linux feature interactions and configuration complexity. Berger et al. [6] studied variability extraction from Kconfig and Kbuild models. In contrast to parser-only or static heuristics, skbuild implements a full symbolic evaluator for Make variable expansions, conditional scoping, and stage-based target aggregation backed by exact Z3 SMT reasoning.

CMake Analysis. Cybolic [1] introduced eager branch-and-merge symbolic execution for CMake build descriptions. skbuild adapts Cybolic's state-merging insight to GNU Make and Kbuild, addressing domain-specific challenges including conditionally-named overwrites, recursive directory traversal, and Kconfig tristate domains.

Variation Representations. Erwig and Walkingshaw's Choice Calculus [10] and Liebig et al.'s variability-aware AST representations [7] formalize software variation, which skbuild's guarded-value mapping instantiates for GNU Make.

6 Conclusion

We presented skbuild, a symbolic-execution engine for Kbuild Makefiles that extracts exact, Z3-verifiable presence conditions for every object file in a configurable software system. By replacing exponential fork-per-branch execution with an eager branch-and-merge symbolic state representation, skbuild scales to real-world codebases without path explosion. Evaluated on BusyBox 1.36.1, coreboot 4.22.01, Barebox 2024.01.0, Das U-Boot 2024.01, and the 1,565-Makefile Linux kernel tree (spanning 2,050 Makefiles and 18,130 targets), skbuild achieves 91.0%–99.5% construct coverage, sub-second to low-second execution across utility and firmware corpora, 22.1 s analysis of the entire Linux tree with under 105 MB peak RSS, 99.0% agreement with real compiler builds across Z3-generated witness configurations, and successfully exposes real-world build anomalies including 6 dead firmware targets in Barebox.

References

- [1] Anonymous Author(s). 2027. Cybolic: Extracting CMake Build Conditions via Symbolic Execution. *Proc. ACM Softw. Eng.*, FSE (2027).
- [2] ThanhVu Nguyen, KimHao Nguyen, and Anh Cuong Phan. 2022. Analyzing the CMake Build System. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '22)*.
- [3] KimHao Nguyen and ThanhVu Nguyen. 2020. Using Symbolic Execution to Analyze Linux KBuild Makefiles. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME '20)*.

- [4] Paul Gazzillo. 2017. Kmax: Analyzing the Linux Kernel Build System with Kbuild and Kconfig. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE '17)*.
- [5] Christian Dietrich, Reinhard Tartler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. Understanding Linux Feature Distribution Through Configuration Complexity. In *Proceedings of the 11th Annual International Conference on Aspect-Oriented Software Development (AOSD '12)*.
- [6] Thorsten Berger, Steven She, Rafael Lotufo, Andrzej Wąsowski, and Krzysztof Czarnecki. 2010. Variability-Aware Parsing in the Presence of Macros and Conditional Compilation. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA '10)*.
- [7] Jörg Liebig, Sven Apel, Christian Lengauer, Christian Kästner, and Martin Schulze. 2010. An Analysis of the Variability in Forty Preprocessor-Based Software Product Lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE '10)*.
- [8] Ahmed Tamrawi, Hoan Anh Nguyen, Hung Viet Nguyen, and Tien N. Nguyen. 2012. Build Code Analysis with Symbolic Evaluation. In *Proceedings of the 34th International Conference on Software Engineering (ICSE '12)*.
- [9] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (1976).
- [10] Martin Erwig and Eric Walkingshaw. 2011. The Choice Calculus: A Representation for Software Variation. *ACM Trans. Softw. Eng. Methodol.* 21, 1 (2011).